ELICA: Efficient and Load Balanced I/O Cache Architecture for Hyperconverged Infrastructures

Mostafa Kishani, Sina Ahmadi, Saba Ahmadian, Reza Salkhordeh, Zdenek Becvar, Onur Mutlu, André Brinkmann, and Hossein Asadi

***Abstract*—*Hyperconverged Infrastructures* (HCIs) combine processing and storage elements to meet the requirements of data-intensive applications in performance, scalability, and quality of service. As an emerging paradigm, HCI should couple with a variety of traditional performance improvement approaches such as I/O caching in virtualized platforms. Contemporary I/O caching schemes are optimized for traditional single-node storage architectures and suffer from two major shortcomings for multi-node architectures: a) imbalanced cache space requirement and b) imbalanced I/O traffic and load. This makes existing schemes inefficient in distributing cache resources over an array of separate physical nodes. In this paper, we propose an *Efficient and Load Balanced I/O Cache Architecture* (ELICA), managing the *solid-state drive* (SSD) cache resources across HCI nodes to enhance I/O performance. ELICA dynamically reconfigures and distributes the SSD cache resources throughout the array of HCI nodes and also balances the network traffic and I/O cache load by dynamic reallocation of cache resources. To maximize the performance, we further present an optimization problem defined by *Integer Linear Programming* to efficiently distribute cache resources and balance the network traffic and I/O cache relocations. Our experimental results on a real platform show that ELICA improves quality of service in terms of average and worst-case latency in HCIs by 3.1× and 23%, respectively, compared to the state-of-the-art.**

***Index Terms*—Hyperconverged infrastructures (HCIs), virtualization, I/O cache, load balancing, quality of service, I/O traffic.**

Received 1 November 2024; revised 13 May 2025; accepted 5 July 2025. Date of publication 24 July 2025; date of current version 2 September 2025. This work was partially funded in part by the Iran National Science Foundation (INSF) under Grant 4025937 and in part by the Research and Technology Deputy of Sharif University of Technology. Recommended for acceptance by D. Feng. *(Corresponding author: Hossein Asadi.)*

Mostafa Kishani is with the Department of Computer Engineering, Sharif University of Technology, Tehran 14588-89694, Iran, and also with the Department of Telecommunication Engineering, Czech Technical University in Prague (CTU), 160 00 Prague, Czechia (e-mail: mostafa.kishani@sharif.edu, kishamos@fel.cvut.cz).

Zdenek Becvar is with the Department of Telecommunication Engineering, Czech Technical University in Prague (CTU), 160 00 Prague, Czechia (e-mail: zdenek.becvar@fel.cvut.cz).

Sina Ahmadi, Saba Ahmadian, and Hossein Asadi are with the Department of Computer Engineering, Sharif University of Technology, Tehran 14588-89694, Iran (e-mail: sina.ahmadi@sharif.edu; saba.ahmadian@sharif.edu; asadi@sharif.edu).

Reza Salkhordeh and André Brinkmann are with the Department of Computer Science, Johannes Gutenberg-University Mainz, 55122 Mainz, Germany (e-mail: rsalkhor@uni-mainz.de; brinkman@uni-mainz.de).

Onur Mutlu is with the Information Technology and Electrical Engineering Department, ETH Zurich, 8092 Zürich, Switzerland (e-mail: omutlu@gmail.com).

Digital Object Identifier 10.1109/TPDS.2025.3592275

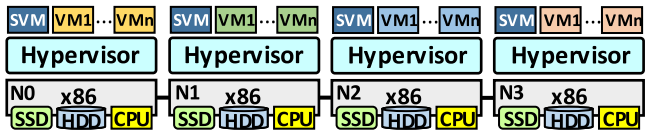


Fig. 1. Example HCI system (VM: Virtual Machine, SVM: Storage Virtual Machine).

I. Introduction

HYPERCONVERGED *Infrastructures* (HCIs) replace legacy data center infrastructures by combining storage arrays, storage networks, and computing servers into an array of single physical nodes, managed under software-defined platforms [30], [69], [87]. As a cost-efficient architecture, HCI finds its applications in many different areas, including *Big Data* [84], *Virtual Desktop Infrastructure* (VDI) [9], *Video Surveillance* [2], and *edge computing* [32]. It is also a promising alternative for applications whose computational resource requirement scales linearly with the storage demand. As the *International Data Corporation* reports, the market has a large tendency to HCI, due to the low provisioning cost and great scalability of HCI platforms.

An HCI architecture with identical nodes is commonly used in enterprise products such as vSAN from VMWare [59], [80], VxRAIL, VxFlex, and XC from Dell EMC [22], [23], [24], SimpliVity from HPE [34], and Acropolis from Nutanix [61]. In HCI, multiple nodes (e.g., four x86 nodes in Fig. 1) are connected to each other, whereas the storage subsystem is distributed over the nodes, providing scalable capacity and performance. Each physical node contains both storage and processing elements managed by a hypervisor hosting a *Storage Virtual Machine* (SVM) to enable storage sharing across all nodes and multiple *Virtual Machines* (VMs). Both storage and performance capacities monotonically scale by adding extra nodes at *no* service downtime.

Despite various advantages, HCI platforms need to couple with a variety of performance improvement paradigms such as I/O caching in virtualized platforms. I/O caching based on *Solid-State Drives* (SSDs) is widely used in enterprise systems [21], [33], [58]. Existing I/O caching schemes, however, are designed to work with legacy storage architectures and new techniques are necessary to adapt caching schemes into emerging HCI platform. VMs in contemporary HCIs are restricted to *only* use the local cache space of their host node, resulting in unfair resource allocation when one node faces a burst of I/O requests.

1045-9219 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

The main challenges of I/O caching in HCI we aim to solve are: **a)** *imbalanced cache space requirement*: while some nodes host VMs with high cache space demands, there exist other nodes hosting less demanding VMs and **b)** *imbalanced I/O load*: some nodes encounter a burst of I/O requests leading to a large cache performance drop, while the rest of the nodes may have idle cache bandwidth. This imbalanced cache management negatively affects *Quality of Service* (QoS) and average I/O latency.

Previous efforts on optimizing the storage subsystem and I/O caching of HCI and multi-node architectures are very limited and are *not* targeted towards optimizing I/O caching across multiple nodes [27], [29], [51], [52], [53], [83], [84], [85], [88]. To the best of our knowledge, *none* of the previous studies have addressed the imbalanced load of I/O caches in HCI. Moreover, the capacity of neighboring HCI node resources in balancing the I/O cache load and decreasing the average and worst-case storage latency has *never* been explored before.

In this paper, we present an *Efficient and Load Balanced I/O Cache Architecture* (ELICA) that provides improved I/O latency and balanced network traffic and I/O cache load in HCI platforms. ELICA distributes the SSD resources within the cluster and shares the total SSD cache space throughout the nodes. To achieve the minimum average latency, we define an optimization problem presented by *Integer Linear Programming* (ILP) that determines the efficient local and remote cache size for each VM. By assigning optimum cache size to active VMs and distributing the I/O cache resources within the cluster, ELICA significantly improves both worst-case and average I/O latency. To reduce the amount of network traffic and to balance the I/O load on SSD cache resources, we propose an online workload characterization scheme in which we manage the distribution of cache resources. In contrast to state-of-the-art load balancing schemes [4], [38] that try to cope with large I/O loads by bypassing the cache and directing requests to the disk subsystem leading to performance loss, ELICA responds to all requests from the cache thanks to its efficient I/O cache distribution across the nodes.

In summary, we offer the following *novel contributions*:

- This paper is the first to propose an efficient I/O caching architecture for HCI, distributing the cache resources throughout the nodes and balancing the load across the SSDs. By online I/O characterization of active VMs, we further propose a novel cache resource distribution scheme, which assigns an efficient cache size for VMs and distributes I/O cache resources within the cluster.
- To achieve the minimum average latency, we define an optimization problem using ILP, which assigns an efficient amount of cache space from local and remote nodes to VMs.
- We propose two cache migration policies, namely, *Instant Migration* and *Gradual Migration* to achieve either low network traffic or low cache space penalty in the cache migration process.
- We implement ELICA and our software-defined HCI module on a real cluster with three physical hardware nodes running Proxmox VE as the hypervisor [64] and Ceph [12] as its backend storage system, and evaluate their efficiency using realistic application workloads.
- We offer a dynamic resource allocation mechanism, which allocates and utilizes the available resources significantly more effectively than state-of-the-art schemes, providing an average of 82% cache utilization on all the workloads tested compared to 34% in the state-of-the-art. Experimental results show that ELICA improves average and worst-case latency by 3.1× and 23%, respectively.

The rest of this paper is organized as follows. Section II discusses the motivation and the experimental setup. We present our proposed architecture in Section III and then detail our I/O cache resource distribution scheme. In Section IV, we discuss the real-system implementation details used in the experiments and then present our experimental results. In Section V, we briefly review the related work and compare them with our proposed architecture. Finally, we conclude the paper in Section VI.

II. Motivation

In this section, we first briefly review I/O caching in HCI. We then setup and conduct a set of experiments on a cluster of three physical servers to demonstrate the I/O imbalance. Finally, we present our motivational results.

A. I/O Caching in HCIs

In existing HCIs, storage elements, namely *disk groups*, are fully controlled by a *Storage Virtual Machine* (SVM). The disk group typically takes advantage of both SSDs and *Hard Disk Drives* (HDDs), where SSDs are used as a caching layer for HDDs [34], [59], [79]. SSDs work as a *global* I/O cache within each node, where the total cache space is shared between the active VMs. However, the I/O cache is *not* distributed within a cluster of HCI nodes; i.e., VMs are restricted to *only* use the local cache space of their hosting node. This architecture can result in imbalanced cache load and cache space usage between HCI nodes hosting VMs with different I/O demands.

Unlike traditional architectures, where compute and storage resources are physically and logically separated, HCI tightly integrates compute and storage into a unified platform. In conventional stacks, compute nodes interact with external storage arrays via network-based protocols (e.g., iSCSI or NFS), which allows independent scaling and optimization. In HCI, however, *Virtual Machines* (VMs) and their associated storage layers (e.g., SSD caches) coexist on the same physical node. This tight coupling introduces several unique performance and management challenges:

- *Resource contention*: Compute, I/O caching, and network traffic contend for shared CPU and memory bandwidth within a node, leading to interference and performance unpredictability.
- *Restricted cache boundaries*: Current HCI caching systems statically allocate SSD cache per node, preventing underutilized nodes from assisting overloaded ones.
- *Costly migrations*: Because compute and storage are co-located, migrating workloads or cache across nodes incurs

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

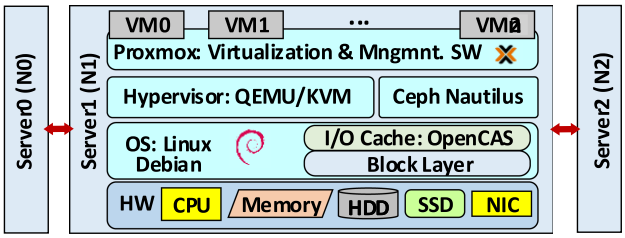


Fig. 2. Implementation stack (hardware and software).

high overhead, unlike traditional virtualization platforms where only compute or data can be moved independently.

Recent studies illustrate the limitations of existing HCI caching strategies:

- Dasarraju et al. [20] observe that current HCI caching setups such as VMware vSAN and Azure Stack HCI statically provision SSD caches per node, with no support for dynamic redistribution—limiting scalability in practice to 16–64 nodes, and potentially leaving remote cache capacity underutilized during imbalanced workloads.
- Shvidkiy et al. [68] evaluate an OpenStack-based HCI platform and demonstrate up to *40% throughput loss* for write-heavy workloads due to contention between storage and compute pipelines on the same node.
- Taheri et al. [75] benchmark VMware vSAN under high VM density and show a *2–3× latency increase* due to inefficient caching and lack of dynamic allocation support.

These findings emphasize the need for intelligent caching mechanisms that adapt to workload imbalance and enable cross-node coordination. ELICA directly targets these shortcomings by supporting cache migration, load-aware allocation, and global cache visibility across nodes. We incorporate these insights in our motivational setup to further illustrate the cache imbalance challenges in real-world HCI deployments.

B. Experimental Setup

Servers Configuration: In our experimental setup, we use a cluster of 2 U rack-mount servers connected with 10GbE network. Each node is equipped with our software-defined HCI platform coupled with the free open-source Proxmox [64] hypervisor (version 6.2). Proxmox is a server virtualization and management software, which integrates the KVM hypervisor and *Linux containers* (LXC), software-defined storage and networking functionality in a single platform [64]. Each node employs 32 Intel(R) Xeon cores, 16 GB Samsung DDR4 DRAM, one Samsung EVO 860 256 GB SSD, and one TB of storage devices (two 500 GB SATA 7.2 K HDDs).

Storage Subsystem: The storage subsystem is built using the open-source software-defined storage platform *Ceph* (version 14.1.0) [12], providing various access types. We setup Intel OpenCAS in our software stack, which is a block-layer I/O cache and is well maintained and supported by the community [8]. We use SSDs as the I/O cache layer for HDDs. The high-level overview of our setup including the hardware and software stack is shown in Fig. 2.

TABLE I
MICROSOFT AZURE APPLICATIONS AND THEIR ASSIGNMENT INTO THE VMS AND HOSTING NODES; (X/Y) DENOTES *TOTAL I/O REQUESTS* OVER *WRITE PERCENTAGE*

Node-0 (N0)	Node-1 (N1)	Node-2 (N2)
VM0 (100/64) VM1 (500/7) VM2 (800/9) VM3 (1K/11)	VM4 (3.5K/8) VM5 (10.6K/99) VM6 (20K/99) VM7 (43K/7) VM8 (55K/6) VM9 (88K/7)	VM10 (230K/20) VM11 (472K/6) VM12 (513K/13) VM13 (1200K/10) VM14 (1300K/9) VM15 (1600K/11) VM16 (1800K/6) VM17 (1900K/10) VM18 (2200K/9) VM19 (2700K/12)

TABLE II
MSR WORKLOADS AND THEIR ASSIGNMENT INTO THE VMS AND HOSTING NODES; (X/Y) DENOTES *TOTAL I/O REQUESTS* OVER *WRITE PERCENTAGE*

Node-0 (N0)	Node-1 (N1)	Node-2 (N2)
VM0:mds_0 (1.2M/12) VM1:rsrch_1 (14K/7) VM2:hm_1 (604K/5) VM3:proj_0 (4.1M/13)	VM4:mds_1 (1.6M/7) VM5:rsrch_2 (206K/34) VM6:src1_0 (37M/44) VM7:proj_1 (23M/11) VM8:stg_0 (2M/85) VM9:prn_0 (5.5M/89)	VM10:rsrch_0 (1.4M/91) VM11:hm_0 (3.9M/65) VM12:src2_0 (1.5M/89) VM13:proj_2 (29M/12) VM14:stg_1 (2.1M/36) VM15:prn_1 (11.1M/25) VM16:usr_0 (2.2M/60) VM17:usr_1 (44M/9) VM18:usr_2 (10M/19) VM19:web_0 (2M/70)

Workloads: Each VM on the cluster (running CentOS 7.2) is equipped with four virtual CPU cores, 2 GB of RAM, and 50 GB of disk space, while the SSD cache space is allocated by ELICA. We first run a subset of Microsoft Azure workloads (detailed in [66] and publicly available on GitHub [55]) using 20 VMs on the HCI platform. Details on the number of VMs alongside the percentage of writes are briefly given in Table I. We also run experiments using a big spectrum of MSR workloads from SNIA [57], [70] (Table II). We assume write-back and *Least Recently Used* (LRU) replacement policies for the I/O cache.

To examine the effectiveness of our architecture in realistic deployment scenarios, we intentionally construct imbalanced workload configurations across the HCI nodes. While we use authentic Azure and MSR traces as our workload sources, we manually control the placement of VMs—assigning I/O-intensive VMs disproportionately to certain nodes (e.g., Node 2), while placing lighter workloads on others (e.g., Node 0). This design reflects common real-world conditions reported in prior work [70], [28], [76], where uncoordinated VM migrations, heterogeneous demands, and dynamic workload shifts lead to load imbalance across nodes in HCI environments.

C. Motivational Results

Quantitative Motivation: To further highlight the limitations of existing cache management strategies in HCI platforms, we conducted additional experiments comparing three approaches: (1) static per-node caching without migration (representing current industry practice, e.g., OpenCAS), (2) a naive round-robin

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

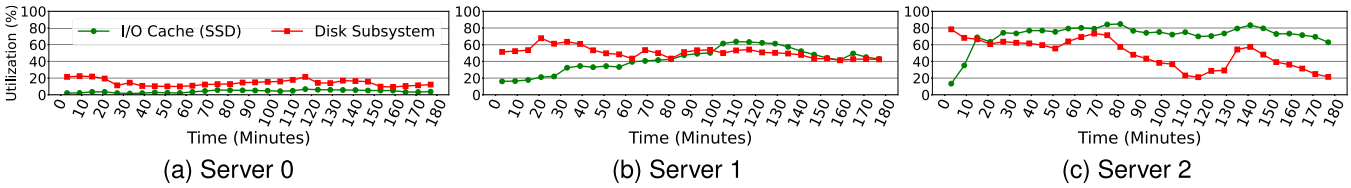


Fig. 3. I/O cache and disk subsystem utilization for a 3-hour run of workloads specified in Table I.

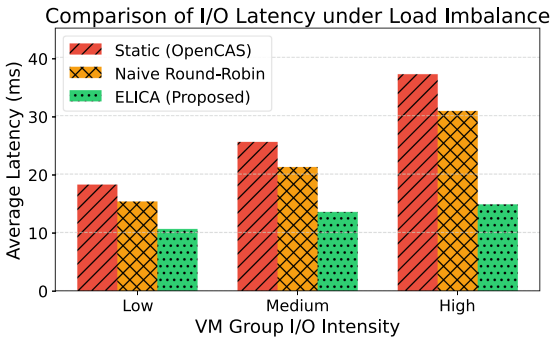


Fig. 4. Latency comparison under different caching strategies and various load conditions.

migration scheme across nodes, and (3) our proposed dynamic, demand-driven allocation with cross-node coordination.

As shown in Fig. 4, static allocation results in up to 2.7× higher average I/O latency compared to ELICA, particularly under skewed workloads where certain nodes face cache saturation. The naive round-robin migration introduces cache thrashing effects, degrading latency by 1.6× relative to ELICA. The mentioned results confirm that existing HCI caching strategies fail to handle workload imbalance and skew effectively, stressing the need for the ELICA adaptive, traffic-aware cache orchestration.

To conduct our motivational experiments, we use the Microsoft Azure dataset. In this dataset, there are 855 distinct applications requesting I/O accesses over a two-week period. Among these applications, there is a variation in how frequently they are invoked. While some of them are invoked once per hour, others are called more or less frequently. The nature of these workloads is mostly read-intensive. We group the applications into three categories and select several applications from each group: a) less than 1 K accesses, b) between 1 K and 100 K accesses, and c) more than 100 K accesses.

To emulate realistic imbalance in VM distribution, we assign the traces to nodes in a non-uniform manner—placing more I/O-intensive VMs on a subset of nodes to simulate skewed cache demand and stress-test the behavior of our system. This controlled imbalance is crucial to evaluate how ELICA mitigates cache pressure through dynamic resource allocation. For the main experimental results (Section IV-B), we conduct the experiments using both workloads (Tables I and II).

Fig. 3 reports the I/O cache (SSD) utilization and disk subsystem (HDD) utilization for the three server nodes in our experimental setup. The utilization is captured in the SSD/HDD device level using Linux *iostat* [36], reporting the device bandwidth utilization, calculated as the percentage of elapsed time during which I/O requests are issued to the device. The I/O cache utilization on Server 0 in Fig. 3 never reaches 10%. The utilization is capped at 60% for Server 1 with some fluctuations. This could be due to the nature of these workloads being more memory-intensive, reading large chunks of data from the storage in different periods and bringing it into the memory. This way, reading from the cache is sometimes beneficial and sometimes not, justifying the fluctuating disk subsystem utilization. Server 2, on the other hand, has a high density of I/O-intensive applications, showing a rise in I/O cache utilization and a decline in HDD utilization in general.

The motivational results serve to show two main challenges of I/O caching in HCI: 1) the imbalanced cache space requirement, and 2) the imbalanced I/O load. As for the first one, for example, in Fig. 3(c) it is seen that the I/O cache utilization reaches to almost 80% during the run, while in Fig. 3(c), the I/O cache is underutilized (less than 10%). This paves the way for an opportunity for using the available cache space on that server, if necessary, to accommodate for workloads with more cache requirements on servers other than the one hosting those specific VMs. As for the I/O load imbalance, one can use the available bandwidth of the I/O cache to serve I/O requests from other nodes. This creates a balanced utilization across all nodes, which helps to improve overall performance. This is what we target to achieve in ELICA, namely, to balance the load of SSDs by allocating remote cache resources from idle nodes to the ones hosting cache-hungry VMs.

III. Proposed Architecture

In this section, we present ELICA and elaborate on how it dynamically partitions the total SSD cache space within the cluster and how it distributes I/O cache resources. We then present how the proposed architecture estimates the cache size of each VM.

A. Overview of ELICA

Fig. 5(a) shows an overview of ELICA. We use HDDs as the main storage media alongside SSDs in the caching layer to address the desired performance at a reasonable cost. To control both hypervisor and shared storage pool, we propose a *Unified Management System* (UMS), which enables configuring the HCI through a central interface. UMS is connected to SVMs and collects information on a) total number of storage elements, and b) allocation of the storage to each individual VM within the cluster. I/O stack management and storage elements handling is also performed by SVM.

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

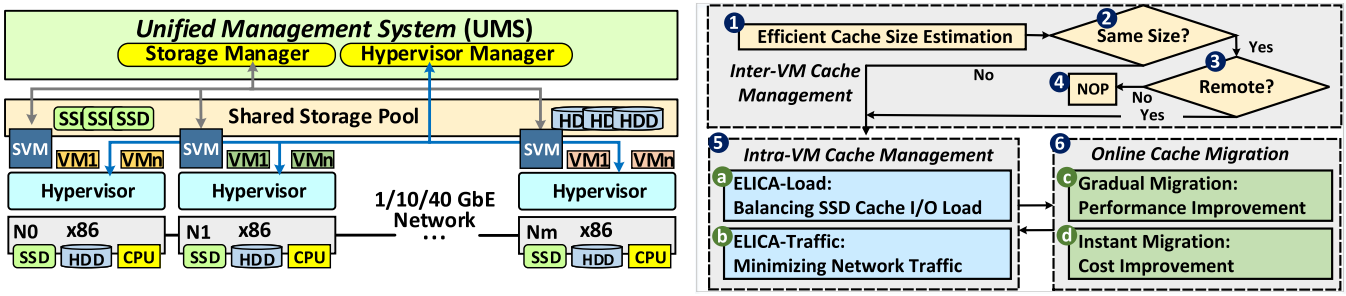


Fig. 5. Overview of ELICA architecture and cache management flow.

The *Storage Manager* of UMS enables ELICA to dynamically partition the total I/O cache space between running VMs and distributes the cache resources through the array of nodes. Fig. 5(b) shows the steps ELICA takes to manage the I/O cache resources across VMs. It has three main phases:

a) Inter-VM Cache Management to Determine Size of Cache Resources: We use ILP to determine the size of cache resource allocated to each VM of the local HCI node as well as of remote HCI nodes. This step provides a mapping table of cache resources to VMs. The objective function of ILP is minimum average latency of storage accesses. According to Fig. 5(b), ① efficient cache size is estimated, ② it is checked whether the current and new cache sizes are the same, ③ in case the allocated cache size is not changed, we check whether remote cache is allocated, ④ in case no remote cache is allocated, no action is required.

b) Intra-VM Cache Management to Determine Cache Chunks of Remotely Allocated VMs: We determine which storage requests of VM are handled using local cache resources and which requests use remote resources (⑤ in Fig. 5(b)). This phase is based on two alternative heuristics, ⓐ trying to reduce the network traffic, called *ELICA-Traffic*, and ⓑ managing to balance the SSD cache load, called *ELICA-Load*. We independently evaluate the performance of both methods for various workloads (Section IV-B). Despite Inter-VM cache management that takes place in larger decision periods, Intra-VM cache manager continuously operates during system runtime.

To distinguish local and remote cache hits, ELICA maintains a global stripe-based address mapping table. Each cache block is tagged with ownership metadata that indicates the node responsible for that stripe. When an I/O request arrives, the UMS checks this mapping table to identify whether the data is stored locally or needs to be fetched from a remote cache. This lookup enables accurate redirection of remote cache hits without requiring global broadcast or excessive coordination overhead. The mapping table is kept in-memory and accessed using constant-time (O(1)) hash lookups, contributing negligible overhead to the I/O path.

c) Cache Migration between HCI Nodes: If either of Inter-VM or Intra-VM cache management phases decide on reallocating a local cache chunk into a remote node (or vice versa), the cache migration is needed (⑥ in Fig. 5(b)). We propose two alternative heuristics for cache migration, independently evaluated for various workloads (Section IV-B), ⓒ *Gradual Migration* and ⓓ *Instant Migration* to achieve either low network traffic or low cost (low cache space penalty).

B. Online Cache Size Estimation

To dynamically partition the SSD cache across active VMs in HCI, we propose an online I/O characterization scheme to determine the performance demands of VMs. We track the I/O accesses of VMs inside the SVM and collect the workload parameters such as request type, destination address, request size, and owner VM. To assign an efficient cache size to VMs assuming LRU policy, we evaluate the LRU *Stack Distance* (SD) [3]. ELICA predicts a cache size for each VM based on stack distance. If the predicted cache space is greater than the physical cache space available on the node, we find the optimal cache size for each VM to achieve the minimum average latency. To do so, we define an optimization problem and present it using ILP.

a) Ideal Cache Size Estimation: SVMs periodically calculate the reuse distance (*LRU Stack Distance*) of the running VMs at the hypervisor they belong to. The output of this step is a set of $SD_{i,j}$ where i and j denote the node (hypervisor) ID and VM ID, respectively. $SD_{i,j}$ is the maximum stack distance in $VM_{i,j}$ [3]. Stack distance of the current storage block access is equal to the number of distinct blocks accessed since the previous access to the current block [3]. Hence, for a cache size greater than stack distance, the current storage block access is a hit. Assuming LRU and a minor change in the workload behavior in a period of the run, by allocating $B > SD_{i,j}$ blocks of cache to $VM_{i,j}$, those blocks accessed in the future are *not* evicted. Hence, $c_{idl_{i,j}}$, the ideal I/O cache size for $VM_{i,j}$ (VM j of node i) is calculated by (1), where $cacheBlockSize$ is set to 8 KB.

$$c_{idl_{i,j}} = (SD_{i,j} + 1) \times cacheBlockSize \tag{1}$$

b) Efficient Cache Size Estimation: UMS collects total I/O cache demands of active VMs from SVMs and allocates the cache space to the VMs throughout the nodes. UMS checks whether the aggregation of estimated ideal cache sizes fits in the cache resources on each node. If the aggregation of ideal cache sizes estimated does not fit in the available cache resources of that node, UMS finds the optimal cache size for each VM by allocating cache resources from both local and remote nodes.

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

TABLE III
DEFINITION OF PARAMETERS USED IN (3)

Parameter	Description
$AVL(VM_{i,j})$	Average latency of $VM_{i,j}$
$c_{i,j}$	Cache size allocated to $VM_{i,j}$
$c_{locali,j}$	Local cache space allocated to $VM_{i,j}$
$c_{remotei,j}$	Remote cache space allocated to $VM_{i,j}$
L_{HDD}	Average read/write latency (i.e., service time) of HDDs
$L_{SSD_{local}}$	Average read/write latency of local SSD cache
$L_{SSD_{remote}}$	Average read/write latency of remote SSD cache
$L_{Network}$	Network latency when remote cache is accessed
$HR(c_{i,j})$	Cache hit ratio of $VM_{i,j}$ from *miss ratio curves* [3], [82]

c) System Model: We consider n_{nodes} disaggregated nodes $N = \{N_1, N_2, \ldots, N_{n_{nodes}}\}$. N_i $(1 \leq i \leq n_{nodes})$ has the local cache capacity of c_{cache_i} and n_{VM_i} virtual machines, where $VM_i = \{VM_{i,1}, VM_{i,2}, \ldots, VM_{i,n_{VM_i}}\}$. The cache space allocated to $VM_{i,j}$ is denoted as $c_{i,j}$. The real workloads have usually very large stack distance for some storage block accesses. Consequently, in practice, the aggregation of ideal cache sizes estimated for each node does not fit in the local cache, hence, (2) is not satisfied. Otherwise, the ideal cache size is allocated to each VM using the local cache of each node. In (2), $c_{idl_{i,j}}$ refers to the ideal cache size of jth VM on the ith node.

$$\forall i \in N : \sum_{j=1}^{n_{VM_i}} c_{idl_{i,j}} \leq c_{cache_i} \tag{2}$$

We ensure that the aggregated cache space allocated to VMs does not surpass the aggregated cache capacity of all nodes. We also assume that the cache space allocated to each VM does not surpass the ideal cache space estimated for that VM. The main objective is to minimize the aggregate *AVerage Latency* (AVL) of VMs, defined as follows (parameters listed in Table III):

$$\begin{aligned} AVL(VM_{i,j}) = HR(c_{i,j}) \times \frac{1}{c_{local_{i,j}} + c_{remote_{i,j}}} \\ \times \left[c_{local_{i,j}} \times L_{SSD_{local}} \right. \\ \left. + c_{remote_{i,j}} \times (L_{SSD_{remote}} + L_{Network})\right] \\ + (1 - HR(c_{i,j})) \times L_{HDD} \end{aligned} \tag{3}$$

d) Problem Definition: We use ILP as an efficient mathematical method to formulate optimization models. The ILP formulation for minimum aggregate AVL is as follows:

$$\min \sum_{i=1}^{n_{nodes}} \sum_{j=1}^{n_{VM_i}} AVL(VM_{i,j})$$

$$\text{s.t.} \sum_{i=1}^{n_{nodes}} \sum_{j=1}^{n_{VM_i}} c_{i,j} \leq \sum_{k=1}^{n_{nodes}} c_{cache_k} \quad \text{(a)}$$

$$c_{i,j} = c_{local_{i,j}} + c_{remote_{i,j}}, \quad \forall i \in N, \forall j \in VM_i \quad \text{(b)}$$

$$c_{remote_{i,j}} = \sum_{k=1, k \neq i}^{n_{nodes}} c_{remote_{i,j_k}}, \quad \forall i \in N, \forall j \in VM_i \quad \text{(c)}$$

TABLE IV
CONSTRAINTS (CS) OF ILP PROBLEM DEFINED IN (4)

Cs	Description
(a)	Sum of all cache sizes assigned to all VMs cannot exceed aggregate cache size available on all nodes.
(b)	VM cache consists of local and remote cache resources.
(c)	VM remote cache is sum of cache resources allocated from all nodes other than hosting node ($c_{remote_{i,j_k}}$ stands for remote cache space allocated from node k (N_k) to $VM_{i,j}$).
(d)	Node cache size is greater or equal to cache space allocated to local hosed VMs and remote VMs hosted by other nodes.
(e)	Cache space of no VM can exceed ideal cache size.

$$\sum_{j=1}^{n_{VM_i}} c_{local_{i,j}} + \sum_{j=1, j \neq i}^{n_{nodes}} \sum_{k=1}^{n_{VM_i}} c_{remote_{j,k_i}} \leq c_{cache_i} \quad \forall i \in N, \quad \text{(d)}$$

$$c_{i,j} \leq c_{idl_{i,j}}, \quad \forall i \in N, \forall j \in VM_i, \quad \text{(e)} \tag{4}$$

The ILP constraints are explained in Table IV. At the first run, cache sizes are evenly distributed across VMs. During each interval *t*, I/O traces are collected by UMS and the information about cache hits are also monitored. After this interval, the stack distance for each VM is calculated using an efficient and optimal algorithm [60] and then ideal cache sizes are determined. If the aggregation of ideal cache sizes does not fit in available local resources, efficient cache sizes are obtained by solving the ILP problem.

C. I/O Cache Resource Distribution

This section presents how ELICA allocates the I/O cache resources, reducing the network traffic and balancing the I/O load of SSD cache devices. We then reconfigure the cache and based on cache demands of VMs, we reallocate the cache space within the nodes or migrate the cache. Distributing I/O cache resources comes with the following: *First*, the allocation of remote cache resources requires communication through the network. This overhead can be critical, especially when we have limited network bandwidth. *Second*, the balanced load between SSD cache devices can help both system performance and SSD lifetime. *Third*, migrating cache resources should be decided based on the network traffic and cache performance. *Fourth*, cache policy and cache coherency should also be decided. These are addressed next in detail. The proposed scheme for the migration of cache resources is distinguished from VM migration, which is occasionally used for maintenance and failure handling [18], [81], [89].

D. ELICA-Traffic: Minimizing Network Traffic

ELICA-Traffic monitors the history of accesses and allocates the local cache to the requests with sequential access pattern.

We prioritize placing sequential workloads in local cache over random ones. This design choice is motivated by two key factors: (1) Sequential workloads tend to generate high-bandwidth, predictable access patterns that are more sensitive to network-induced latency and congestion. Keeping them local ensures

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

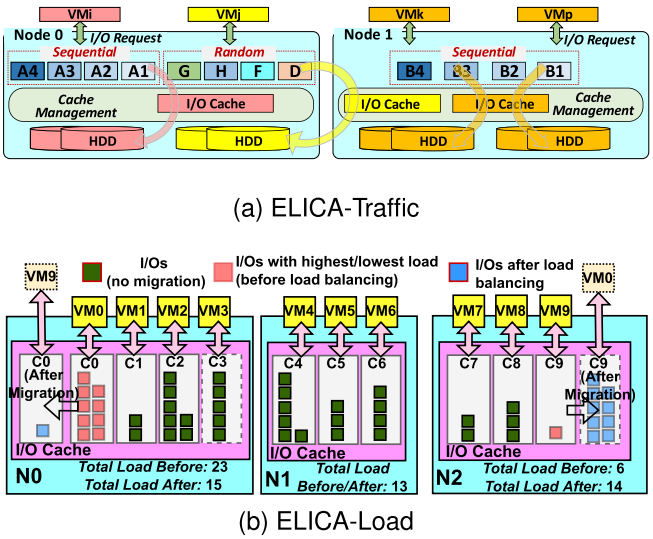


Fig. 6. Overview of ELICA-Traffic and ELICA-Load.

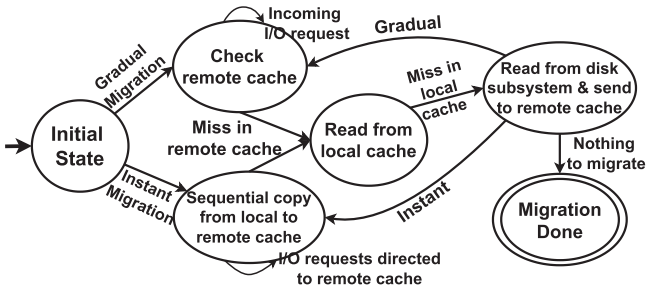


Fig. 7. State diagram of ELICA migration schemes.

lower access latency and avoids additional network hops. (2) Caching sequential data locally reduces cache fragmentation, as these accesses exhibit higher spatial locality and can utilize cache space more efficiently.

Our experimental results (Fig. 12) confirm that prioritizing sequential requests in local cache improves average latency by **23%** and reduces worst-case tail latency under both MSR and Azure workloads compared to demand-driven allocation policies.

Compared to random workloads, sequential accesses impose higher bandwidth overhead to the network (which is similar to large-size I/O). To reduce the network traffic, ELICA-Traffic allocates the cache space from the local SSDs. If the local resources are not sufficient, we allocate local cache to the sequential requests while the cache for random accesses is allocated from remote nodes (Fig. 6(a)). ELICA-Traffic recognizes a sequence of requests to consecutive addresses as sequential, when the aggregate size of requests surpasses a threshold (64 KB in our experiments). The sequential requests use the local cache resources until a change in the pattern is recognized.

E. ELICA-Load: Balancing SSD Cache Load

The purpose of ELICA-Load is balancing SSD cache load within the HCI in a way that the capacity of cache allocated to each VM does not change. The main idea is to exchange cache chunks between busy and idle nodes to balance the SSD cache load. To this end, we first estimate the load on the SSDs. We consider *Queue Depth* (QD), the number of waiting requests in each SSD queue, as a representative for SSD load. A large QD leads to higher latency. ELICA selects n cache chunks from Node A with the maximum load (QD) and exchanges them with n cache chunks from Node B, which has the least load. This way, the load on the busy node is decreased and, consequently, waiting time becomes shorter and the overall performance will improve. Note that n cache chunks of Node A (which has the maximum load) are selected from the VM with the maximum load, while n cache chunks of Node B are selected from the VM with the minimum load.

Fig. 6(b) shows an exemplary HCI with three nodes and the load on each VM cache. Using the load balancing scheme, ELICA decides to exchange the cache with the highest load (C0) from the busy node (Node-0) with the cache of the lowest load (C9) from the least-busy node (Node-2). Doing so, we reduce the load on Node-0 by 35% (from 23 down to 15) while the load on Node-2 increases from 6 to 14.

F. Online I/O Cache Migration

ELICA migrates the cache of a VM from the hosting node to a remote node for two reasons: a) cache space of the local node is not sufficient and b) to balance SSD load. We propose an online migration scheme by considering three key parameters affecting the migration cost: 1) *migration time*, 2) *reserved capacity*, and 3) *bandwidth* imposed on SSDs and network during migration. *Migration time* denotes the duration of moving the VM cache from $node_i$ to $node_j$. *Reserved capacity* stands for the cache space allocated to the VM in the previous location. Once the migration is accomplished, the reserved capacity is released. *Bandwidth* denotes the I/O traffic imposed on the source SSD (due to reading data blocks), destination SSD (due to writing migrated data blocks), and network.

To mitigate the migration cost, ELICA employs two migration schemes: 1) *Gradual Migration* (GM) and 2) *Instant Migration* (IM). GM aims to reduce the *bandwidth* requirements while increasing *migration time* and imposing high *reserved capacity* due to long migration time. In contrast, IM targets minimum *migration time* and aims to reduce *reserved capacity* overhead while consuming high *bandwidth* and imposing greater I/O load compared to GM. Fig. 7 depicts the migration schemes using a state diagram.

1) Gradual Migration (GM): GM migrates cache blocks once they are referenced by a read or write access. To gradually migrate an I/O cache resource from one node to another, we first allocate the required cache space (reserved capacity) in the destination SSD and then perform the following. *First*, all accesses to the storage blocks are directed to the remote cache. *Second*, we check the request type and in the case of write access, we check whether the request hits in the cache or not. In the case of a cache miss (i.e., in both local and remote caches), the

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

access (for writes) is served by the remote cache. Otherwise, in the case of a cache hit in the local cache, the request is served by the remote cache and the corresponding block in the local cache is invalidated. *Third*, in case of read access and upon a cache hit, the request is handled through the remote cache. Otherwise, in case of cache miss, we check whether the request can be served by the local cache or not. In the case of a hit in the local cache, we serve the request by the local cache and also migrate the corresponding block to the remote cache. Finally, in case of a miss in the local cache, the request is served by the disk subsystem and the data is promoted to the remote cache.

GM provides two key benefits: 1) it reduces the number of accesses to the source SSD and 2) it prevents serving the accesses including write, RAW, eviction, and promotion from the local cache while only read hits are responded through the local cache. GM is not efficient regarding *reserved capacity*. It may take a long time while we need to dedicate the cache *reserved capacity* in both local and remote caches. We propose an alternative method, called *Instant Migration* (IM) offering an efficient *reserved capacity* at a bandwidth penalty.

2) Instant Migration (IM): IM aims to aggressively migrate the cache resources to minimize the *migration time* and enhance *reserved capacity* while imposing a high *bandwidth* requirement and I/O load. To instantly migrate a cache resource from $node_i$ to $node_j$, we first allocate the required space in $node_j$. We then take the following steps. *First*, we start reading the cache blocks sequentially from the local cache and writing into the remote one. We then invalidate the migrated cache blocks in the local cache. *Second*, the I/O access is first routed to the remote cache and in case of a miss, we check the content of the local cache. If the data block hits in the local cache, we respond it, move the block to the remote cache, and invalidate the local cache. *Third*, in case of a miss in both the remote and local caches, we respond from the disk subsystem and promote the data block to the remote cache. Using IM, we can migrate the I/O cache quickly and release the *reserved capacity* in the local node as soon as possible.

3) Network Traffic Considerations: While ELICA allows redirecting I/O requests to remote caches, this redirection introduces additional network traffic, particularly under write-intensive or bursty workloads. To mitigate network traffic overheads, ELICA integrates two complementary mechanisms:

- *Rate-limited Migration:* ELICA limits the number of cache migrations initiated per unit time, ensuring that network bandwidth used for migration does not overwhelm the data plane. This prevents sudden spikes in traffic that could interfere with VM I/O requests.
- *Latency-aware Migration Selection:* Migration decisions incorporate awareness of the current network congestion state. During periods of high congestion, ELICA employs Gradual Migration to spread data movement over time, while Instant Migration is used during low-congestion periods to expedite data redistribution.

These mechanisms ensure that remote cache access and migrations introduce minimal disruption to network performance, while still enabling the benefits of dynamic, load-balanced cache allocation.

G. Cache Policy and Cache Coherency

There are generally two types of caching policies, namely, *write-through* and *write-back*. In write-through, data is persisted synchronously in both the SSD cache and disk subsystem. Write-back works asynchronously and keeps the modified data temporarily in the SSD cache while destaging dirty data blocks to the disk subsystem later. One consideration when choosing a cache policy is the workload type. To reduce write traffic and in order to gain maximum improvement in terms of throughput and latency, we choose write-back as our caching policy [44].

Another important parameter is cache coherency, which refers to handling data stored on multiple caches and ensuring the consistency of data across the system. In our proposed I/O caching, coherency could potentially be threatened in the following scenario. When allocating remote cache resources to VMs and dynamic reallocation of cache at runtime which needs data migration, it is possible that data blocks in the migration phase are changed in the application layer. This issue can be studied in three situations.

a) When data has not been migrated yet: The VM request for changing a block or blocks of data is first applied and then the migration starts.

b) When data is being migrated to the destination node: After receiving a request, the source node running the application is responsible to notify the destination node on the change and provide it with the new data. The destination node then invalidates incoming data and replaces it with the new one.

c) When data has been migrated to the destination node: The request will be directed to the remote cache (as discussed in Section III-F). Since the data has been fully placed inside the remote cache and invalidated in the local one, there will be only one copy of the data and coherency is thus preserved.

Instant Migration and Update Safety: In ELICA, both Instant Migration (IM) and Gradual Migration (GM) adhere to the same three-phase consistency mechanism to ensure that data is never moved while being updated. Specifically, during *Instant Migration*, ELICA ensures that:

- All in-flight or pending writes to the cache block are flushed and acknowledged before migration starts.
- Any write arriving during the transfer is intercepted at the source, tracked, and resent to the destination after migration is completed.
- Only after all blocks are safely received and confirmed at the destination node, ELICA updates the global mapping and invalidates the local copy.

This design ensures that even in Instant Migration—which performs migration aggressively without throttling—*data is never moved concurrently with active writes*. Migration only starts after confirming that no write conflicts are present, guaranteeing data correctness.

IV. Experimental Results

A. Implementation Details

Here, we detail our software stack implemented on physical servers. We first discuss the employed open-source tools and

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

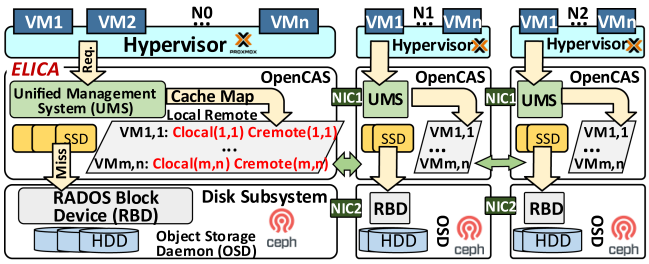


Fig. 8. High-level implementation schema.

the changes made to accommodate our needs. Then, we detail the UMS implementation. Fig. 8 shows a high-level view of our implementation for a three-node cluster.

1) Hypervisor: There are several choices when it comes to selecting a hypervisor. One can use a full-fledged hypervisor such as ESXi. While providing many features, such hypervisors are not free. Since we aim to run UMS (details in Section IV-A4) inside the hypervisor, while also keeping the cost at a minimum, we opted for a free and open-source hypervisor. We chose Proxmox [64], which is a virtualization platform based on *Debian* and has built-in features to integrate with *Ceph*.

2) Storage Subsystem: Choosing a platform for the backend storage is a critical decision as it should provide: 1) *scalability:* to scale out the nodes easily without any downtime or performance degradation, 2) *fault tolerance:* to ensure data accessibility using various techniques such as replication, 3) *compatibility:* to provide access to data in different formats. Ceph is an open-source distributed storage system, which is purely software-defined and has all three features. It uses *object storage* as its underlying layer but has the ability to provide file, block, and object level access through its interfaces. We use Ceph *Rados Block Device* (RBD), which provides block level access to data.

3) I/O Caching Mechanism: In order to build the I/O caching layer, we chose OpenCAS [8], as it is well maintained and updated. OpenCAS is a block caching software module that accelerates access to main storage. Using OpenCAS, we are also able to collect statistics such as cache hit ratio at a low memory and CPU overhead (less than 10% of a CPU core utilization per HCI node). Parameters such as caching mode and line size can easily be configured. In our experiments, we chose the write-back policy with 8 KB cache line size. To integrate OpenCAS in ELICA, we made small changes to its source code. As detailed in Section III-C, it is required to access certain blocks of data belonging to an arbitrary VM in the I/O cache. For this purpose, about 100 lines of code were added to OpenCAS so that the required data could be retrieved from the I/O cache.

4) Unified Management System (UMS): UMS is made up of four modules written using C++ and Python in about 8 K lines of code and runs inside the hypervisor. These modules communicate with each other using *User Datagram Protocol* (UDP) packets, suited for low-latency applications. They maintain a map of current cache sizes (local and remote) for VMs.

a) I/O Request Capture: It is responsible for taking care of incoming I/O requests. Each VM running inside the hypervisor has a unique *Process ID* (PID) assigned. By using Linux tools such as *blktrace* [10], it can identify the source of each incoming request and make a queue of the requests based on the time they are issued at the VM level. Blktrace has a very low overhead, less than 2% as reported in [1], [50], not seriously affecting the overall system performance.

b) Dynamic Cache Allocation: Cache stats from previous run are used to find out whether current cache sizes need to be changed by calculating their new *stack distance* for each configurable interval. New remote cache sizes that need to be accommodated by another node are communicated with the corresponding node UMS. The destination node allocates the specified amount of cache space for a VM residing in another node, updates the map, and then notifies the other nodes.

c) Local/Remote Handling: On distribution of I/O cache resources, cluster network and SSD load are two main concerns addressed in ELICA. Based on the method used, this module handles allocation of local or remote cache to requests. E.g., reads have higher priority for local cache.

d) Stats Collector: Cache statistics such as hit and miss ratios are collected for each individual VM to be used in the next run by the Dynamic Cache Allocation module.

5) Workloads: We use two widely adopted real-world traces in our evaluation: the Microsoft Azure VM I/O traces and the MSR Cambridge enterprise workload suite. These workloads represent realistic, production-level I/O behavior observed in cloud and enterprise HCI deployments. Specifically:

- *MSR traces* include I/O patterns from various server classes (e.g., web, development, project servers), capturing diverse access patterns, temporal locality, and request sizes [57].
- *Azure traces* capture VM-level I/O activity in production cloud environments, reflecting multi-tenant, bursty workloads typical of virtualized infrastructures [11].

These traces are commonly used in systems research (e.g., [19], [25], [35], [39], [63], [86]) to benchmark caching, scheduling, and storage policies under heterogeneous and imbalanced loads. Their diversity and realism make them especially suitable for evaluating dynamic cache allocation in HCI systems like ELICA.

B. Experimental Results

In this section, we evaluate ELICA and show how it effectively manages the I/O cache resources in HCI and improves worst-case and average latency. We compare ELICA with the state-of-the-art hypervisor-based caching scheme, ECI-Cache [3], in which the cache resources of each node are locally managed (denoted as ECI). We implement and evaluate both ELICA and ECI-Cache on a physical HCI platform for both Microsoft Azure and MSR workloads (Tables I and II). We choose our monitoring interval to be 10 minutes, meaning that after this interval, the new cache stats are collected, the ILP is solved in the background, and the new cache sizes are determined for the next interval. A smaller monitoring interval translates to a more accurate cache resource allocation at the cost of a higher computation overhead. The efficient length of monitoring interval, however, is related to the frequency of changes in the workloads. We examine all workloads with different monitoring intervals and realize that

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

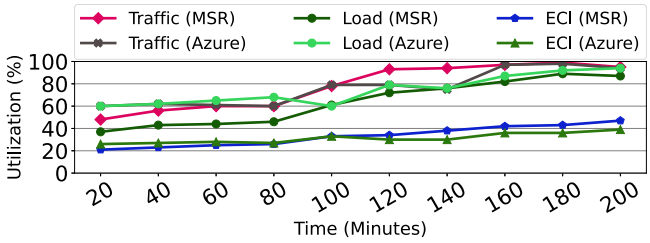


Fig. 9. Cache Utilization.

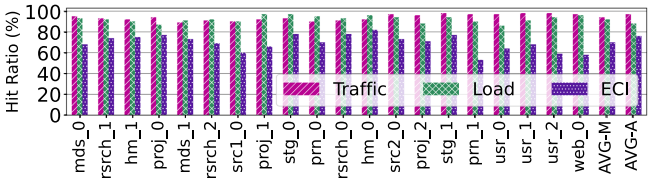


Fig. 10. Hit ratio (AVG-M and AVG-A are average over MSR and Azure workloads, respectively).

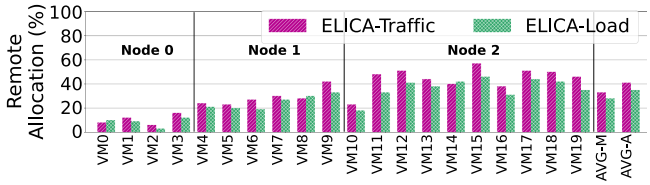


Fig. 11. Percentage of remote cache allocated.

the ILP accuracy does not have a considerable improvement for monitoring intervals smaller than 10 minutes.

1) Cache Resource Allocation: Here we compare cache allocation of ELICA and ECI, residing in the hypervisor. We use GM to move I/O cache resources. Fig. 9 shows the cache resource usage for each method for MSR and Azure workloads. We make the following major observations. ELICA increases the I/O cache allocation by 2.7× compared to ECI for MSR and 2.4× for Azure workloads. This is thanks to our global resource allocation. VMs in ECI are restricted to *only* use the cache of the local nodes. Hence, some nodes face the lack of cache space while some other nodes have idle cache space. Considering the average of workloads on all three nodes, ELICA demonstrates 76% and 70% higher cache resource utilization compared to ECI for MSR and Azure workloads, respectively.

To better compare performance results, Fig. 10 compares the hit ratios of ELICA-Traffic and ELICA-Load schemes as well as ECI. By efficient cache allocation, ELICA shows 33% and 35% higher hit ratio compared to ECI for the MSR and Azure workloads, respectively. This improvement was expected since in both ELICA-Traffic and ELICA-Load, more cache resources are allocated to VMs. An important observation is the greater impact of ELICA on heavy workloads. For example, for prn_0 (on VM9), which is an I/O intensive workload, the improvement of ELICA in both Traffic and Load schemes is approximately 76%. This shows when local cache is not enough, remote cache allocation is very beneficial.

2) I/O Cache Resource Distribution: We examine ELICA to show how the proposed resource manager effectively distributes the VM cache resources across the HCI array. ELICA aims to use both local and remote SSDs for cache allocation, achieving an efficient resource distribution when the demands of nodes have a large variance. Fig. 11 shows the percentage of remote resources allocated by ELICA-Load and ELICA-Traffic. We can make the following major observations. ELICA-Load allocates 37% remote resources, while ELICA-Traffic allocates 31% remote resources on both MSR and Azure workloads. To reduce the I/O load on the local SSDs, ELICA-Load moves the VMs cache resources to remote nodes. Meanwhile, ELICA-Traffic that targets reducing the network traffic aims to allocate fewer number of remote resources to VMs.

VMs running on *Node 0* have the least amount of remote cache allocated to them (Fig. 11). Since the server hosts fewer VMs running on this node, and except for one workload (proj_0 on VM3), the other three are less I/O intensive, it rarely faces lack of cache space and it can accommodate all the requests using its local cache. Another observation is the higher amount of remote cache allocated to write-heavy workloads. For example, VM12 is running src2_0, which is write-intensive and it has a high amount of remote cache.

3) Performance Improvement: We compare the performance of ELICA-Load and ELICA-Traffic with ECI in terms of worst-case and average latencies of active VMs using *iostat* [36]. Compared to ECI, ELICA achieves higher hit ratio and lower latency since it distributes the I/O caches among the nodes. Hence, VMs can use the free space of SSDs in the remote nodes. Besides, ELICA balances I/O load in the HCI array. In case of high I/O load on the nodes hosting VMs with high access frequency, the cache resources are moved to the nodes with idle SSD bandwidth.

4) Average Latency Improvement: Fig. 12(a) compares the average latency of ELICA-Load and ELICA-Traffic with ECI. Using GM and IM on MSR workloads, and compared to the ECI based HCI, ELICA-Load improves the average latency by 2.76× (176%), while ELICA-Traffic shows a 3.37× (237%) latency improvement. These numbers are 2.83× for ELICA-Load and 3.44× for ELICA-Traffic on Azure workloads. This can be explained by the fact that ECI only allocates local cache and fails to provide efficient amount of cache to the VMs, leading to more frequent HDD accesses. For a workload such as N0-VM3 (running proj_0), ELICA mostly allocates local cache with very minimal remote cache and is able to achieve almost 1.9× latency improvement over ECI. The significant improvement is due to more cache space released by moving highly loaded I/O caches to other nodes. Workloads that are assigned more remote cache tend to have higher average latency compared to workloads with more local cache. This latency, however, is still much lower than the case where all workloads on a server use only the local cache available on that server (e.g., ECI), exhausting its cache resources.

5) Worst-Case Latency Improvement: Fig. 12(b) and (c) compare the worst-case latency achieved by ELICA and ECI based HCI for the 99.9th and 99.99th percentiles, respectively. The average results over both MSR and Azure workloads are also

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

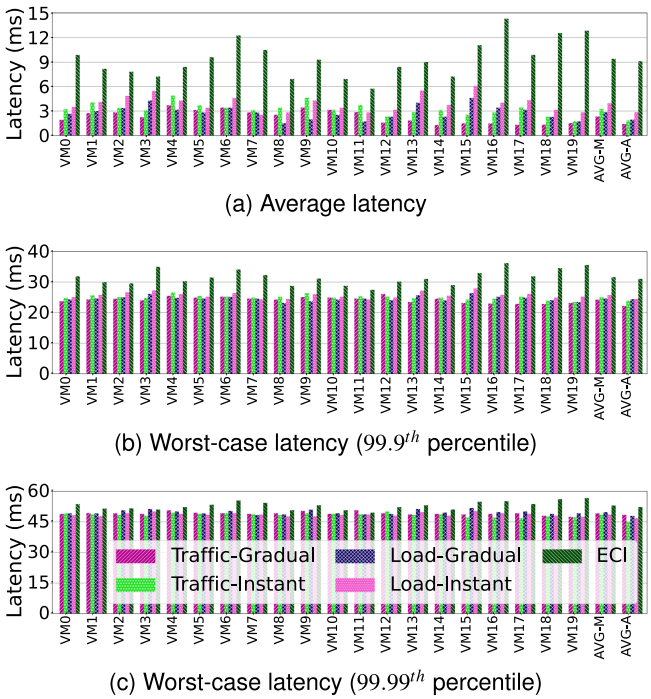


Fig. 12. GM vs. IM: average and worst-case latency (AVG-M: average over MSR, AVG-A: average over Azure).

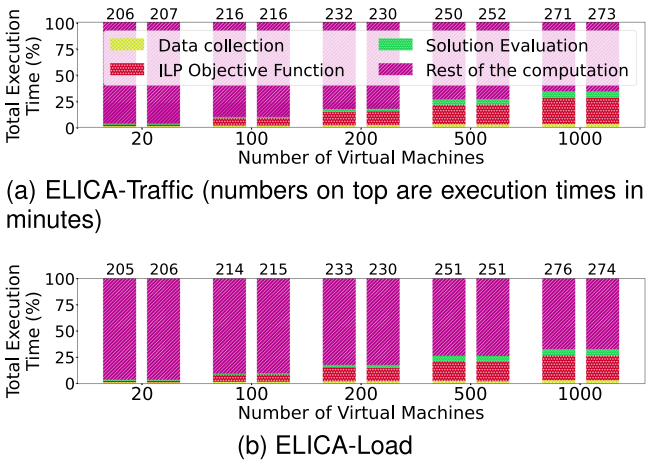


Fig. 13. ILP overhead (Left bar: GM and right bar: IM).

shown in this figure. ELICA-Load and ELICA-Traffic achieve an average worst-case latency improvement of 23% and 13% for three and four 9's, respectively. Moving from three-nine to four-nine percentile, the tail latency for both ELICA methods seems to get closer to ECI. This is because worst-case latency is highly correlated to HDD queue depth. As these parameters increase, the waiting time for I/O requests also increases, resulting in longer latency. It is also the case that remote cache may not be available to be assigned to a VM cache at some point (due to reasons such as cache space used up by other VMs). This will cause ELICA to act very similar to ECI, allocating and using only the local cache.

6) Per-Node Throughput Under Varying Load: To evaluate the ELICA scalability and performance under varying stress levels and application profiles, we measured I/O throughput across nodes under three load conditions, using workloads representative of real-world HCI applications.

Workload Profiles:

- *OLTP-like workload (similar to TPC-C pattern):* Simulates small, random, write-heavy transactional workloads, representative of database backends.
- *OLAP-like workload (similar to TPC-H pattern):* Simulates large, sequential, read-intensive analytical queries.
- *Mixed workload (enterprise-like file server pattern):* Emulates a mix of sequential and random I/O with varying read/write ratios.

Load Profiles and Mapping to Applications:

- *Low Load:* Nodes primarily run OLAP-like VMs with low I/O frequency, emulating off-peak analytical workloads.
- *Moderate Load:* Nodes host a balanced mix of OLTP, OLAP, and mixed workload VMs, reflecting typical enterprise activity.
- *High Load:* Nodes run multiple OLTP-like VMs with aggressive random I/O, simulating peak transactional demand.

Results: Fig. 14 shows the per-node aggregate throughput (in MB/s) under each scenario:

- Under *low load*, Nodes 0–2 achieve throughput between 280–300 MB/s, reflecting optimal cache usage and minimal queuing with read-dominant traffic.
- Under *moderate load*, throughput remains stable at 260–270 MB/s, as the ELICA adaptive cache allocation ensures balanced performance across diverse I/O types.
- Under *high load*, throughput drops to 215–230 MB/s due to SSD queuing pressure from write-heavy, random OLTP-style workloads. However, the degradation is controlled, as the ELICA traffic-aware balancing and migration prevent bottlenecks.

These results, illustrated in Fig. 14, confirm that ELICA maintains robust throughput and scalability across varying application-level workloads and system stress conditions, validating its ability to support realistic, end-to-end HCI applications.

7) ILP Computational Overhead: In ELICA, we solve ILP in each 10-minute interval. By limiting the search space and defining bounds using *Branch and Bound* method, we can solve ILP with a reasonable time and memory complexity. To show the ILP overhead, we perform computations using different number of VMs. Fig. 13 shows what percentage of the total execution time solving ILP takes. The execution times (in minutes) for each run is demonstrated on top of each bar. To evaluate the ILP overhead, we stop the system and run the ILP offline and compare it with the concurrent system run. ILP calculation consists of three main phases. First, all information regarding cache sizes are collected from different nodes. Second, the ILP objective function is solved and new cache sizes are obtained. Third, the most recent ILP solution is evaluated. The results show the overhead is by 18% of the total execution time when 500 VMs run in the cluster. ILP computations are executed using *only* a single thread on one of the servers. While ILP is not memory-intensive, it is compute-intensive, imposing overhead

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

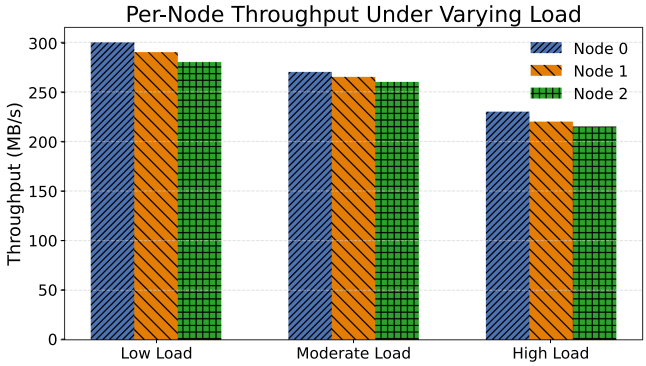


Fig. 14. Per-node throughput under three different load scenarios.

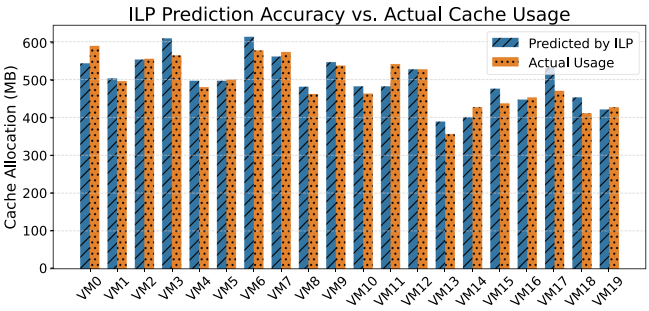


Fig. 15. ILP prediction accuracy: per-VM predicted cache size vs. actual cache usage.

on a single CPU core. This overhead can be evenly distributed over multiple CPU cores by extending our implementation to a multi-threaded program. Fig. 18 shows the CPU overhead of ILP in a single-thread implementation. Increasing the ILP computation interval is another approach to reduce this overhead. Besides, as HCI is mostly targeted for small to medium-size deployments, we conclude that the ILP computation overhead is negligible and can easily be compensated by either increasing time interval or multi-threaded programming.

Our choice of ILP is motivated by its structured modeling of system constraints and the availability of efficient solvers that return near-optimal solutions using hybrid optimization techniques (e.g., branch-and-bound, heuristics). While ILP is NP-hard, our workload-aware formulation remains lightweight and solvable within seconds even at large VM scales. The solver operates asynchronously in the control plane and is decoupled from the I/O path, making it a practical choice for dynamic cache allocation.

We acknowledge that alternative approaches such as heuristic algorithms or *Machine Learning* (ML) models could also be explored. However, for this work, we prioritized explainability and minimal runtime dependencies. ML-based cache management, especially using reinforcement learning, remains a promising direction for future research.

8) ILP Accuracy and System Impact: ILP Prediction Accuracy. To evaluate how well ILP models real-world cache demands, we compared the cache sizes predicted by the solver to the actual usage measured at runtime. As shown in Fig. 15, the majority of predicted allocations closely match observed usage.

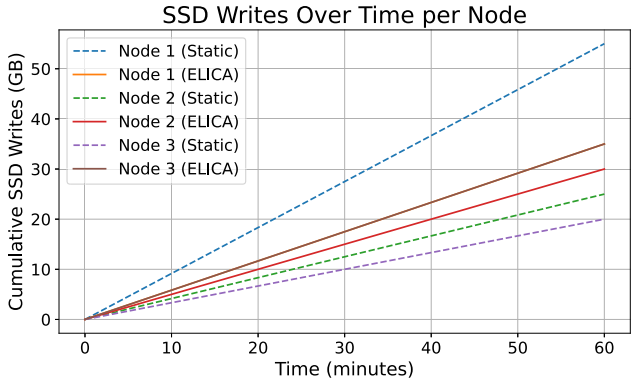


Fig. 16. Cumulative SSD writes over time under skewed load. Static policy overuses Node 1, while ELICA distributes writes more evenly.

- The *Mean Absolute Error* (MAE) between predicted and actual cache usage is **24.3 MB**, which is low given that average per-VM allocations are around 512 MB.
- **85% of VMs** experienced cache size prediction errors within **10%** of actual usage.

9) SSD Endurance and Wear-Leveling: SSD endurance is primarily affected by write amplification and sustained write volume. ELICA mitigates these effects through a combination of traffic-aware migration and adaptive cache allocation:

- *Write-aware migration:* Gradual migration spreads writes over time, avoiding sudden bursts that can prematurely degrade flash cells. This leads to more uniform wear leveling across devices.
- *Workload-aware placement:* Write-heavy workloads are not preferentially allocated to a single node. Instead, cache space is distributed based on traffic patterns, avoiding overuse of any specific SSD.
- *Observed workload balance:* In our experiments, workloads exhibited a wide range of read/write ratios (6%–85% writes across VMs). ELICA adapts to this heterogeneity and ensures no node becomes a hotspot for write activity.

To quantify the impact of ELICA on SSD endurance, we track cumulative SSD writes per node under skewed load. As shown in Fig. 16, static cache partitioning results in a significantly higher write volume on Node 1. In contrast, ELICA redistributes the load, leading to a more balanced write profile across all nodes.

We also estimate SSD wear by comparing the percentage of P/E cycle budget consumed across nodes. As shown in Fig. 17, the static policy results in uneven wear, with Node 1 consuming 45% of its P/E budget. ELICA maintains uniform wear levels at approximately 30% per node.

10) Network Bandwidth Utilization Across Servers: The main overhead of ELICA on the network is transmitting data blocks. To monitor network usage, we use *IPTraf* [37]. Fig. 19 shows the total network bandwidth utilization for three servers in the experiments. We only report usage for ELICA related communication while leaving out the usage for non-ELICA processes. The network utilization in our study does not go beyond 40% in total (14%, on average).

11) GM Versus IM: We first redo our previous experiments using IM to make a comparison on performance in terms of

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

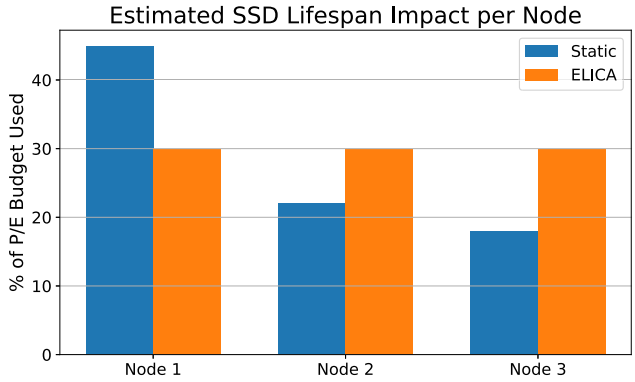


Fig. 17. Estimated SSD wear (% of P/E budget used per node). ELICA achieves better wear leveling across the cluster.

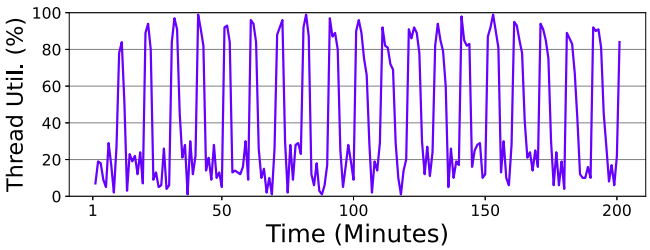


Fig. 18. Utilization of single-thread implementation calculating ILP over time.

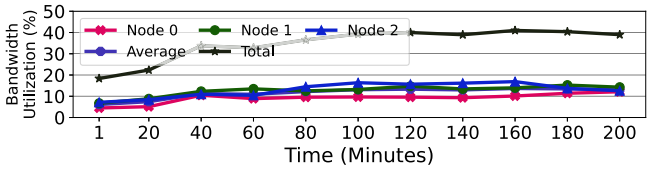


Fig. 19. Servers Bandwidth Utilization.

average and worst-case latency. We then perform experiments to compare the performance of our proposed migration schemes in terms of *bandwidth* and *migration time* to show possible trade-offs.

Fig. 12 compares the average and worst-case latency achieved using two migration schemes. For the average latency, IM is about 23% faster (Fig. 12(a)), while for the worst-case latency, we observe a 10% improvement. We expected to see better latency results for IM since it aims to quickly migrate cache resources to the destination node, while GM only migrates when a corresponding request is issued. This causes an overhead in searching as ELICA needs to search both local and remote caches to find the desired data blocks. Although this can also happen in IM, it would not last for a long time compared to GM, hence resulting in lower latency.

Fig. 20(a) shows the total amount of time spent and bandwidth used for two migration schemes of three sample workloads. These workloads have the most remote cache resources allocated (Fig. 11). Using GM, it takes a maximum of 14 minutes to migrate local cache to the other nodes for *prn_1* (VM15). The migration happens at a bandwidth of up to 7 MB/s, while using IM finishes the migration in less than two minutes at up to 223 MB/s bandwidth (7× faster but with 31× higher bandwidth overhead than GM). The same scenario is valid for other two workloads. To summarize, IM provides faster migration at a higher bandwidth while GM takes bandwidth into account and does the migration at a slower pace.

TABLE V
CACHE CONSISTENCY VALIDATION UNDER SYNTHETIC MIGRATION TESTS

Metric	Instant	Gradual
Migrations tested	10,000	10,000
Corrupted blocks detected	0	0
Integrity verification success	100%	100%

To ensure data correctness during migration, we run 10,000 synthetic migration events under write-heavy workloads. Each migrated block is verified using SHA-256 checksums. Table V summarizes the results for both gradual (GM) and instant (IM) migration. In all runs, we observed zero block corruption, confirming that ELICA preserves cache consistency.

V. RELATED WORKS

In this section, we show why previous schemes cannot be effectively applied to the existing HCIs. Several studies [40], [41], [42], [43], [67], [72], [73] developed various schemes to enhance the reliability of storage arrays and HCIs while *none* of them takes the performance improvement of HCIs into account. Table VI summarizes contribution domains of previous works.

A. I/O Caching in Virtualized Platforms

Several SSD-based I/O caching schemes are proposed for virtualized platforms, improving performance, while considering endurance and reliability [3], [4], [6], [38], [45], [49], [54] (Table VI). Contemporary I/O caching schemes for virtualized platforms mainly focus on partitioning the total SSD cache space between running VMs to maximize the performance. Such schemes, however, are optimized to single host structures. They are also not compatible with HCIs as they lead to imbalanced I/O load and cache space allocation (as discussed in Section IV-B). A group of studies also manage to estimate an efficient cache size for virtualized platforms [3], [6], [45], [49], [54] using parameters such as *working set size*, locality, and reuse intensity of the running workloads. Few studies (e.g., CloudCache [6]) propose to migrate the entire VM (not only the I/O cache) to other hosts in case of insufficient cache space. Recent work [47] has explored machine learning-based approaches for fair and efficient cache management in NVMe SSDs, demonstrating the potential for intelligent caching strategies in virtualized environments.

B. Load Balancing in Distributed / Storage Systems

Several schemes migrate high-load VMs to nodes having less load [7], [16], [62]. However, costs associated with live migration of VMs could be prohibitive in practical deployments of HCI such as edge computing [17], [71], [90]. SIB [38], LBICA [4], and DistCache [48] are three recent I/O cache load balancing schemes. In case of burst and heavy I/O accesses, both LBICA and SIB balance the load of the SSD cache by bypassing the requests into the disk subsystem. Hence, the requests that

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

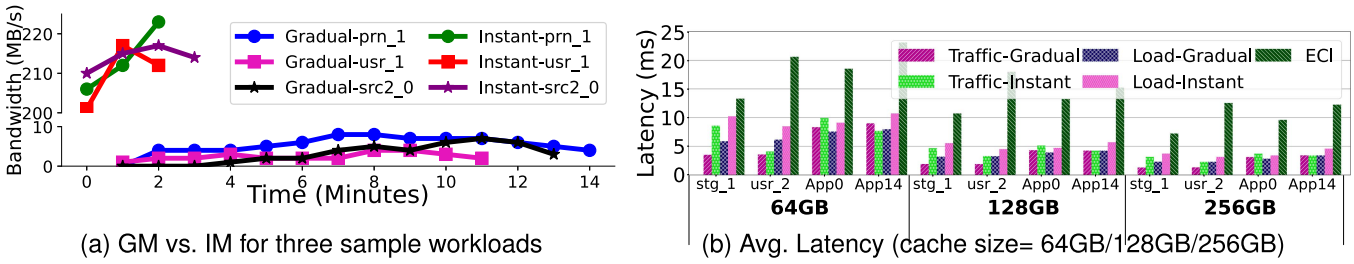


Fig. 20. Comparison of (a) Gradual vs. Instant Migration schemes and (b) average latency.

TABLE VI
DOMAIN OF CONTRIBUTIONS IN PREVIOUS WORKS

System Type / Contributions	Cache/VM Migration	Load Balancing	Unified Cache	Cache Size Alloc.	Reliability/Endurance/Coherency
HCI	ELICA	ELICA	ELICA	ELICA	[73], [74], [67]
Distributed Storage/Virtualized	[77], [5], [18]	[65], [7], [16], [62], [48]		[6]	[6], [42], [4]
Serverless		[66], [83]	[66], [83], [29]	[66], [83], [29]	[83]
Single Storage/Virtualized		[38], [4], [31], [88]		[45], [3], [49], [54]	[41], [43], [42], [40], [45], [3], [49], [54]

are considered to be responded by the cache (hit) experience the high latency of disk subsystem, which leads to considerable I/O performance drop. These schemes also do not take the distributed architecture of HCIs into account.

In [15], the intersection of data deduplication and migration strategies have been explored as a way to achieve load balancing in distributed storage systems, highlighting the complex interplay between data management and system performance.

Recent studies have proposed another category of cache management techniques to improve fairness among competing applications accessing shared SSDs [46], although they use DRAM as cache.

C. I/O Cache Migration

Online migration of data across data centers in a geo-distributed manner is addressed in [77], where *distributed storage overlays* are introduced to cache data objects and facilitate data migration. Akkio [5] is a locality management service, deciding on when and how to perform data migration regarding the data access footprint, reducing the response time and resource usage. Clark et al. [18] investigate the live migration of entire VMs, which is completely different from I/O cache migration proposed in ELICA. VM migration comes with downtime in the operation or serious performance degradation and has a large migration time and network traffic, only used for occasions such as failures, maintenance, and hardware/software updates [18], [81], [89].

D. I/O Caching in Commercial Hypervisors

While commercial hypervisors [56], [78], [79] support I/O caching in both guest and host file-systems, they do *not* support distributed I/O caching. The deprecated VMware flash read cache (known as VFlash), as well as a third-party cache software such as *Virtunet* [74] developed to integrate with VMware, only use the local host resources. VMware vSAN [78], [79] also uses local cache resources in both all-flash and hybrid HCI solutions.

E. I/O Caching in Alternative Platforms

Remero et al. [66] proposed a distributed cache for *Function-as-a-Service* (FaaS) applications. The service paradigm in FaaS applications is based on microservice containers that have a different software stack from VMs in HCIs. The work presented in [66] is useful for DRAM cache management, while ELICA manages SSD I/O cache. Hence, the contribution of [66] is not necessarily applicable to HCI platforms. In a similar effort, Wang et al. [83] propose a cache memory for FaaS application, relying on DRAM cache and a distributed architecture. To address cold start problem in FaaS applications, Fuerst and Sharma [29] propose FaasCache that keeps frequently used FaaS functions alive after they have finished execution. None of these studies use cache migration as a facility to balance the cache load. Farshin et al. [27] explore the use of *Direct Cache Access* (DCA) facility in Intel processors to improve packet processing latency in *Software-Defined Networks* (SDNs). Wang et al. [85] also explore DCA for SDNs in a recent study. Ge et al. [31] use I/O access hints in favor of I/O latency. Wu et al. [88] propose a caching mechanism for data storage systems, adapting to the characteristics of emerging storage media. As both works focus on a single data storage node, they do not deal with challenges such as imbalanced load between the nodes.

F. Discussions and Future Work

a) Mirrored I/O Cache: One future direction of this research is distributing the mirrored cache into two different HCI nodes. When caching the write-pending data, it can be written into two cache modules for the sake of performance and reliability. ELICA makes room for distributing the mirrors into two different HCI nodes rather than a single node, doubling the read bandwidth of the local I/O cache. While this scheme can improve the average read latency, the overhead of addressing cache coherency upon the write requests should be explored.

b) Endurance of I/O Cache: Data migration may have an impact on the endurance of SSD devices, depending on the selected migration mechanism. In GM, no extra cache write

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

is imposed, as the cache is gradually migrated by redirecting new write operations. IM, however, comes with write overheads, as it aggressively migrates all cached data to the new destination.

c) I/O Cache versus Page Cache: In ELICA, we mainly focused on the I/O cache of HCI block storage that resides in the I/O block layer, just below the page cache. The page cache and I/O cache work independently in different layers of Linux storage stack, while optimizations in each layer can independently help overall storage performance [13], [14], [26].

d) Reliability: Cache coherency concerns are fully addressed in ELICA, as discussed in Section III-G. Both ELICA and ECI use SSD in the caching layer, resulting in similar reliability obligations. By employing mirrored I/O cache, ELICA outperforms ECI in the case of motherboard and/or system failures, which is single-point-of-failure in the ECI architecture. Nevertheless, a higher reliability level can be achieved only when data blocks are also replicated in multiple nodes.

e) I/O Cache Size: To show the efficiency of ELICA with smaller cache sizes (64 GB and 128 GB), we perform experiments on four sample workloads (VM0 and VM14 chosen from Table I and stg_1 and usr_2 chosen from Table II). Our experiments show ELICA improves average latency by 2.5× (151%), 3.4× (241%), and 3.5× (253%) over 64 GB, 128 GB, and 256 GB cache sizes, respectively, compared to ECI. ELICA also improves worst-case latency by 26%, 30%, and 31% over 64 GB, 128 GB, and 256 GB cache size, respectively (detailed results are removed for the sake of brevity). Further analysis and use of heterogeneous cache size on different nodes is another future direction for this research.

f) All Methods in One: While in this study we examined ELICA-Traffic and ELICA-Load separately, combining both ELICA-Traffic and ELICA-Load methods is left for future work, as they seem to have a co-dependency. As an example, in trying to balance the load, we need to accept some traffic overheads and vice versa.

g) Comparison with Commercial Systems: We compare ELICA against ECI-Cache [3], a prior hypervisor-based caching system that supports per-VM cache partitioning and workload-aware allocation. Its architecture closely aligns with our goals, enabling a meaningful technical comparison. In contrast, commercial platforms such as VMware vSAN, OpenShift, and Ignite, differ significantly in design: vSAN offers static, node-local caching without per-VM migration; OpenShift targets containerized storage without block-level cache control; and Ignite requires application-level integration. These differences make direct, fair comparison infeasible. Future work will explore integrating ELICA as a pluggable cache orchestrator into such environments.

h) Relationship to In-Memory Caching Systems: Distributed in-memory caches such as Apache Ignite, Memcached, and Redis are designed for application-level data acceleration, typically requiring explicit integration by developers. Distributed in-memory caches, cache the structured data (e.g., key-value pairs) in RAM to reduce access latency. In contrast, ELICA operates at the storage layer of HCI platforms, transparently managing SSD-based block-level cache for virtual machines. Unlike in-memory caches, ELICA requires no application modifications and addresses system-level concerns such as cache migration, load imbalance, and endurance. The two approaches are complementary: in-memory caching improves data access at the application tier, while ELICA improves storage I/O performance at the infrastructure tier.

VI. Conclusion

In this paper, we presented an I/O caching architecture for HCIs, called ELICA, which allocates efficient cache spaces to VMs and effectively distributes the I/O cache resources throughout the HCI array, improving the QoS in terms of average and worst-case latency. To effectively partition the SSD cache space across active VMs, ELICA estimates an efficient cache size for each VM based on workload characteristics and distributes the I/O cache resources across the HCI array. To this end, an ILP optimization problem is presented. To reduce the performance overhead imposed by migrating the I/O cache, we proposed two online cache migration schemes, namely, Gradual and Instant Migration. The first migration scheme benefits network traffic, while the latter one manages to achieve the best cache space efficiency by migrating the I/O cache as soon as possible. Our experiments on a real platform revealed that ELICA improves the QoS in term of average and worst-case latency by 3.1× and 23%, respectively.

References

[1] Block I/O layer tracing using blktrace, *Smackerel of Opinion*, 2009, Accessed: Oct. 08, 2022. [Online]. Available: https://smackerelofopinion.blogspot.com/2009/10/block-io-layer-tracing-using-blktrace.html
[2] Pivot3 HCI solutions, *Pivot3*, 2021, Accessed: Apr. 12, 2021. [Online]. Available: https://pivot3.com/solutions/
[3] S. Ahmadian, O. Mutlu, and H. Asadi, "ECI-Cache: A high-endurance and cost-efficient I/O caching scheme for virtualized platforms," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 2, no. 1, pp. 9:1–9:34, Apr. 2018.
[4] S. Ahmadian, R. Salkhordeh, and H. Asadi, "LBICA: A load balancer for I/O cache architectures," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2019, pp. 1196–1201.
[5] M. Annamalai et al., "Sharding the shards: Managing datastore locality at scale with Akkio," in *Proc. 13th USENIX Symp. Operating Syst. Des. Implementation*, 2018, pp. 445–460.
[6] D. Arteaga et al., "CloudCache: On-demand flash cache management for cloud computing," in *Proc. 14th Usenix Conf. File Storage Technol.*, 2016, pp. 355–369.
[7] E. Arzuaga and D. R Kaeli, "Quantifying load imbalance on virtualized enterprise servers," in *Proc. 1st Joint WOSP/SIPEW Int. Conf. Perform. Eng.*, 2010, pp. 235–242.
[8] R. Baldyga, Opencas, 2018. [Online]. Available: https://github.com/open-cas/open-cas-linux/
[9] A. Bednarz, "What is hyperconvergence?," *Network World*, 2022, Accessed: Apr. 12, 2022. [Online]. Available: https://www.networkworld.com/article/3207567/what-is-hyperconvergence.html
[10] Blktrace, Blktrace: Block layer IO tracing tool. 2006. [Online]. Available: https://linux.die.net/man/8/blktrace
[11] B. Calder et al., "Windows Azure storage: A highly available cloud storage service with strong consistency," in *Proc. 33rd ACM Symp. Operating Syst. Princ.*, 2011, pp. 143–157.
[12] Ceph, "Ceph: An open-source, distributed storage system," *Official Ceph Platform, first launched c.2012*, Last Accessed: Jan. 01, 2022. [Online]. Available: https://ceph.io/en/
[13] H.-P. Chang, Y.-C. He, and D.-W. Chang, "An integrated memory and SSD caching I/O subsystem," in *Proc. IEEE 42nd Annu. Comput. Softw. Appl. Conf.*, 2018, pp. 823–824.

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

[14] H.-P. Chang, C.-N. Liao, and D.-W. Chang, "A page cache management scheme in cloud computing environments," in *Proc. IEEE Int. Conf. Dependable, Autonomic Secure Comput., Int. Conf. Pervasive Intell. Comput., Int. Conf. Cloud Big Data Comput., Int. Conf. Cyber Sci. Technol. Cong.*, 2019, pp. 974–979.

[15] G. Cheng, L. Luo, J. Xia, D. Guo, and Y. Sun, "When deduplication meets migration: An efficient and adaptive strategy in distributed storage systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 10, pp. 2749–2766, Oct. 2023.

[16] H. WonH. ChoiA. KwakSohn, and K. Chung, "Autonomous learning for efficient resource utilization of dynamic VM migration," in *Proc. 22nd Annu. Int. Conf. Supercomputing*, 2008, pp. 185–194.

[17] A. Choudhary, M. C. Govil, G. Singh, L. K. Awasthi, E. S. Pilli, and D. Kapil, "A critical survey of live virtual machine migration techniques," *J. Cloud Comput.*, vol. 6, no. 1, pp. 1–41, 2017.

[18] C. Clark et al., "Live migration of virtual machines," in *Proc. 2nd Conf. Symp. Netw. Syst. Des. Implementation*, 2005, pp. 273–286.

[19] M. Copik, R. Böhringer, A. Calotoiu, and T. Hoefler, "FMI: Fast and cheap message passing for serverless functions," in *Proc. 37th Int. Conf. Supercomputing*, 2023, pp. 373–385.

[20] V. K. Dasarraju, "Implementing and managing hyper-converged infrastructure with VMware vSAN and Azure stack HCI," *J. Artif. Intell. Mach. Learn. Data Sci.*, vol. 2, no. 2, pp. 666–670, 2024.

[21] EMC Dell, Dell EMC Unity: FAST Technology Overview, *Dell Technologies*, 2018. [Online]. Available: https://www.delltechnologies.com/asset/en-us/products/storage/industrymarket/h15086-emc-unity-fast-technology-overview.pdf

[22] EMC Dell, DELL EMC VXRACK FLEX Product Overview, Accessed: Aug. 3, 2025. [Online]. Available: https://www.dell.com/enus/shop/ipovw/vmware-vxrack

[23] EMC Dell, Dell EMC VXRail Appliance TechBook, 2019. Accessed: Aug. 3, 2025. [Online]. Available: https://www.delltechnologies.com/asset/ru-ru/products/convergedinfrastructure/technical-support/h15104-vxrail-appliance-techbook.pdf

[24] Dell EMC and Intel, "*Dell EMC XC Series Advancements and Vision for Customers in the Hyperconverged Era*", 2019. Accessed: Aug. 3, 2025. [Online]. Available: https://www.dell.com/en-sg/dt/converged-infrastructure/xcseries/index.htm#tab0=0

[25] T. Eldeeb, X. Xie, P. A. Bernstein, A. Cidon, and J. Yang, "Chardonnay: Fast and general datacenter transactions for {On-Disk} databases," in *Proc. 17th USENIX Symp. Operating Syst. Des. Implementation*, 2023, pp. 343–360.

[26] Z. Fan and D. Park, "Extending SSD lifespan with comprehensive non-volatile memory-based write buffers," *J. Comput. Sci. Technol.*, vol. 34, no. 1, pp. 113–132, 2019.

[27] A. Farshin, A. Roozbeh, G. Q. Maguire Jr., and D. Kostic, "Reexamining direct cache access to optimize I/O intensive applications for multi-hundred-gigabit networks," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 673–689.

[28] J. B. Fernandes, Í. A. S. de Assis, I. Martins, T. Barros, and S. Xavier-de Souza, "Adaptive asynchronous work-stealing for distributed load-balancing in heterogeneous systems," 2024, *arXiv:2401.04494*.

[29] A. Fuerst and P. Sharma, "FaasCache: Keeping serverless computing alive with greedy-dual caching," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Virtual Event, USA, 2021, pp. 386–400.

[30] S. Gakhar and V. K. Nirwal, "Device replacement for hyper-converged infrastructure computing environments," US Patent App. 15/684, 965, 2018.

[31] X. Ge, Z. Cao, D.H. C. Du, P. Ganesan, and D. Hahn, "Hintstor: A framework to study I/O hints in heterogeneous storage," *ACM Trans. Storage*, vol. 18, no. 2, pp. 1–24, 2022.

[32] N. Hassan, A. K.-L. Yau, and C. Wu, "Edge computing in 5G: A review," *IEEE Access*, vol. 7:127276–127289, 2019.

[33] HP, HPE Smart Array SR SmartCache, 2018. Accessed: Aug. 3, 2025. [Online]. Available: https://buy.hpe.com/us/en/software/server-management-software/servermanagement-software/smart-array-management-software/hpe-smart-array-srsmartcache/p/5364342

[34] HPE, "*HPE SimpliVity Hyperconverged Infrastructure for VMware vSphere*," 2019. Accessed: Aug. 3, 2025. [Online]. Available: https://www.hpe.com/psnow/doc/a00019351enw

[35] I. Iliadis, "Reliability evaluation of erasure-coded storage systems with latent errors," *ACM Trans. Storage*, vol. 19, no. 1, pp. 1–47, 2023.

[36] IOSTAT, IOSTAT. 2018. [Online]. Available: https://linux.die.net/man/1/iostat

[37] IPTraf, IPTraf. Accessed: Apr. 2022. [Online]. Available: https://github.com/iptraf-ng/iptraf-ng

[38] J. Kim, H. Roh, and S. Park, "Selective I/O bypass and load balancing method for write-through SSD caching in Big Data analytics," *IEEE Trans. Comput.*, vol. 67, no. 4, pp. 589–595, Apr. 2018.

[39] T. Kim et al., "Morph: Efficient file-lifetime redundancy management for cluster file systems," in *Proc. ACM SIGOPS 30th Symp. Operating Syst. Princ.*, 2024, pp. 330–346.

[40] M. Kishani, S. Ahmadian, and H. Asadi, "A modeling framework for reliability of erasure codes in SSD arrays," *IEEE Trans. Comput.*, vol. 69, no. 5, pp. 649–665, May 2020.

[41] M. Kishani and H. Asadi, "Modeling impact of human errors on the data unavailability and data loss of storage systems," *IEEE Trans. Rel.*, vol. 67, no. 3, pp. 1111–1127, Sep. 2018.

[42] M. Kishani, R. Eftekhari, and H. Asadi, "Evaluating impact of human errors on the availability of data storage systems," in *Proc. Conf. Des., Automat. Test Europe*, European Design and Automation Association, 2017, pp. 314–317.

[43] M. Kishani, M. Tahoori, and H. Asadi, "Dependability analysis of data storage systems in presence of soft errors," *IEEE Trans. Rel.*, vol. 68, no. 1, pp. 201–215, Mar. 2019.

[44] R. Koller, L. Marmol, R. Rangaswami, S. Sundararaman, N. Talagala, and M. Zhao, "Write policies for host-side flash caches," in *Proc. 11th USENIX Conf. File Storage Technol.*, 2013, pp. 45–58.

[45] R. Koller, A. J. Mashtizadeh, and R. Rangaswami, "Centaur: Host-side SSD caching for storage performance control," in *Proc. IEEE Int. Conf. Autonomic Comput.*, 2015, pp. 51–60.

[46] R. Liu, Z. Tan, L. Long, Y. Wu, Y. Tan, and D. Liu, "Improving fairness for SSD devices through DRAM over-provisioning cache management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2444–2454, Oct. 2022.

[47] W. Liu, J. Cui, T. Li, J. Liu, and L. T. Yang, "A space-efficient fair cache scheme based on machine learning for NVME SSDs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 383–399, Jan. 2023.

[48] Z. Liu et al., "Distcache: Provable load balancing for large-scale storage systems with distributed caching," in *Proc. 17th USENIX Conf. File Storage Technol.*, 2019, pp. 143–157.

[49] T. Luo, S. Ma, R. Lee, X. Zhang, D. Liu, and L. Zhou, "S-cave: Effective SSD caching to improve virtual machine storage performance," in *Proc. 22nd Int. Conf. Parallel Architectures Compilation Techn.*, 2013, pp. 103–112.

[50] S. Mehnaz and E. Bertino, "Ghostbuster: A fine-grained approach for anomaly detection in file system accesses," in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy*, New York, NY, USA, 2017, pp. 3–14.

[51] C. Melo et al., "Models for hyper-converged cloud computing infrastructures planning," *Proc. Int. J. Grid Utility Comput.*, vol. 11, no. 2, pp. 196–208, 2020.

[52] C. Melo et al., "Availability models for hyper-converged cloud computing infrastructures," in *Proc. Annu. IEEE Int. Syst. Conf.*, 2018, pp. 1–7.

[53] S. Meneses, E. Maya, and C. Vasquez, "Network design defined by software on a hyper-converged infrastructure. case study: Northern technical university fica data center," in *Proc. Int. Conf. Syst. Inf. Sci.*, Springer, 2020, pp. 272–280.

[54] F. Meng, L. Zhou, X. Ma, S. Uttamchandani, and D. Liu, "vCacheShare: Automated server flash cache space management in a virtualization environment," in *Proc. USENIX Annu. Tech. Conf.*, 2014, pp. 133–144.

[55] Microsoft, "Azure functions blob access trace," 2020. Accessed: Jun. 2022. [Online]. Available: https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsBlobDataset2020.md

[56] Microsoft, "*Hyper-V storage caching layers and data consistency requirements*," Microsoft Learn, May 25, 2021, Accessed: Aug. 3, 2025. [Online]. Available: https://learn.microsoft.com/en-us/previousversions/troubleshoot/windows-server/hyper-v-storage-caching-layers-dataconsistency-requirements

[57] D. Narayanan, A. Donnelly, and A. Rowstron, "Write off-loading: Practical power management for enterprise storage," *ACM Trans. Storage*, vol. 4, no. 3, 2008, Art. no. 10.

[58] NetApp, "SSD cache feature," 2017. [Online]. Available: https://library.netapp.com

[59] J. Nicholson and P. Koehler, "A Day in the Life of a VSAN I/O–Diving in to the I/O Flow of vSAN," *VMworld 2016 Session STO7875 Summary, PenguinPunk.net Blog*, Sep. 2016. [Online]. Available: https://www.penguinpunk.net/blog/vmware-vmworld-2016-sto7875-a-day-in-the-life-of-a-vsan-io/

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.

[60] Q. Niu, J. Dinan, Q. Lu, and P. Sadayappan, "Parda: A fast parallel reuse distance analysis algorithm," in *Proc. IEEE 26th Int. Parallel Distrib. Process. Symp.*, 2012, pp. 1284–1294.
[61] Nutanix, "Hyperconverged infrastructure: The definitive guide," *NUTANIX*, 2022. [Online]. Available: https://www.nutanix.com/go/what-is-nutanix-hyperconverged-infrastructure
[62] J.-G. Park, J.-M. Kim, H. Choi, and Y.-C. Woo, "Virtual machine migration in self-managing virtualized server environments," in *Proc. 11th Int. Conf. Adv. Commun. Technol.*, 2009, pp. 2077–2083.
[63] J. G. V. Pauloski et al., "Object proxy patterns for accelerating distributed applications," *IEEE Trans. Parallel Distrib. Syst.*, 2024.
[64] Proxmox, "Proxmox VE Administration Guide," Proxmox Server Solutions GmbH, Accessed: Dec. 26, 2021. Available: https://pve.proxmox.com/pve-docs/pve-admin-guide.html
[65] D. Puthal et al. Y, "Secure and sustainable load balancing of edge data centers in fog computing," *IEEE Commun. Mag.*, vol. 56, no. 5, pp. 60–65, May 2018.
[66] F. Romero et al., "Faa $ t: A transparent auto-scaling cache for serverless applications," in *Proc. ACM Symp. Cloud Comput.*, Seattle, WA, USA, 2021, pp. 122–137.
[67] R. Shekar, M. Vanninen, G. Costea, R. Carter, and E. Chiu, "Hyperconverged infrastructure supporting storage and compute capabilities," US Patent App. 15/431, 525, 2018.
[68] A. A. Shvidkiy, A. V. Spirkina, A. A. Savelieva, and A. V. Tarlykov, "Evaluation of the impact the hyper-converged infrastructure storage subsystem synchronization on the overall performance," in *Proc. 12th Int. Cong. Ultra Modern Telecommun. Control Syst. Workshops*, 2020, pp. 248–252.
[69] G. U. Siddappa, S. Nayak, R. K. R. Kottapalli, S. S. Hemige, and S. Verma, "Methods and apparatus to manage compute resources in a hyperconverged infrastructure computing environment," US Patent App. 16/133, 742, 2019.
[70] SNIA, "Microsoft Enterprise Traces," *Storage Networking Industry Association (SNIA) IOTTA Repository*, 2022. Accessed: Aug. 3, 2025, [Online]. Available: https://iotta.snia.org/traces/block-io/130
[71] G. Soni and M. Kalra, "Comparative study of live virtual machine migration techniques in cloud," *Int. J. Comput. Appl.*, vol. 84 no. 14, pp. 19–25, 2013.
[72] S. Srinivasan, D. Reed, and D. Cummins, "Hybrid hyper-converged infrastructure and storage appliance," US Patent 9, 830,0 82, 2017.
[73] S. Srinivasan, D. Reed, and D. Cummins, "Hyper-converged infrastructure based on server pairs," US Patent 9, 778, 865, 2017.
[74] Virtunet Systems, "Host-side caching software virtucache," 2023. Accessed: Aug. 3, 2025. [Online]. Available: https://virtunetsystems.com/host-side-caching-software-virtucache/
[75] H R. Taheri, G. Little, B. Desai, A. Bond, D. Johnson, and G. Kopczynski, "Characterizing the performance and resilience of HCI clusters with the TPCx-HCI benchmark," in *Perform. Eval. Benchmarking Era Artif. Intell.: 10th TPC Technol. Conf.*2018, Rio de Janeiro, Brazil, Springer, 2019, pp. 58–70.
[76] W.-H. Tian, M.-X. Xu, G.-Y. Zhou, K. Wu, C.-Z. Xu, and R. Buyya, "Prepartition: Load balancing approach for virtual machine reservations in a cloud data center," *J. Comput. Sci. Technol.*, vol. 38, no. 4, pp. 773–792, 2023.
[77] N. Tran, M. K. Aguilera, and M. Balakrishnan, "Online migration for geo-distributed storage systems," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2011, pp. 273–284.
[78] VMWare, "VSAN disk groups," *VMware Cloud Foundation Blog*, Apr. 18, 2019. Accessed: Aug. 3, 2025. [Online]. Available: https://blogs.vmware.com/cloud-foundation/2019/04/18/vsandisk-groups/.
[79] VMWare, "An Overview of VMware Virtual SAN Caching Algorithms," *VMware Cloud Foundation Blog*, Apr. 18, 2019. Accessed: Aug. 3, 2025. [Online]. Available: https://blogs.vmware.com/cloud-foundation/2019/04/18/vsan-diskgroups/#:~:text=Cache%20Tier,-As%20mentioned%20the%20capacity%20tier
[80] VMWare, "VMWare VSAN 6.7 Technical Overview," 2019. Accessed: Aug. 3, 2025. [Online]. Available: https://www.vmware.com/docs/vmware-vsantechnology-overview
[81] W. Voorsluys, J. Broberg, S. Venugopal, and R. Buyya, "Cost of virtual machine live migration in clouds: A performance evaluation," in *Proc. IEEE Int. Conf. Cloud Comput.*, Springer, 2009, pp. 254–265.
[82] C. A. Waldspurger, N.A. ParkGarthwaite, and I. Ahmad, "Efficient MRC construction with SHARDS," in *Proc. File Storage Technol.*, 2015, pp. 95–110.
[83] A. Wang et al., "Infinicache: Exploiting ephemeral serverless functions to build a cost-effective memory cache," in *Proc. 18th USENIX Conf. File Storage Technol.*, Santa Clara, CA, USA, 2020, pp. 267–281.
[84] J. C. Wang and B. Kim, "Automotive Big Data pipeline: Disaggregated hyper-converged infrastructure vs hyper-converged infrastructure," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 1784–1787.
[85] M. Wang, M. Xu, and J. Wu, "Understanding I/O direct cache access performance for end host networking," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 1, pp. 22:1–22:37, 2022.
[86] Z. Wang et al., "Ransom access memories: Achieving practical ransomware protection in cloud with {DeftPunk}," in *Proc. 18th USENIX Symp. Operating Syst. Des. Implementation*, 2024, pp. 687–702.
[87] J. Wu, K. R. Gurram, and K. Kattumadam, "Systems and methods for policy driven storage in a hyper-convergence data center," US Patent App. 10/078, 465, 2018.
[88] K. Wu et al., "The storage hierarchy is not a hierarchy: Optimizing caching on modern storage devices with orthus," in *Proc. 19th USENIX Conf. File Storage Technol.*, 2021, pp. 307–323.
[89] F. Xu, F. Liu, H. Jin, and A. V. Vasilakos, "Managing performance overhead of virtual machines in cloud computing: A survey, state of the art, and future directions," *Proc. IEEE*, vol. 102, no. 1, pp. 11–31, Jan. 2014.
[90] F. Zhang, G. Liu, X. Fu, and R. Yahyapour, "A survey on virtual machine migration: Challenges, techniques, and open issues," *IEEE Commun. Surv. Tuts.*, vol. 20, no. 2, pp. 1206–1243, Second Quarter 2018.

Authorized licensed use limited to: CZECH TECHNICAL UNIVERSITY. Downloaded on October 21,2025 at 10:59:46 UTC from IEEE Xplore. Restrictions apply.